Dynamic Programming Study Guide

What is Dynamic Programming?

Dynamic Programming (DP) is an algorithmic technique for solving optimization problems by breaking them down into simpler subproblems. It stores the results of subproblems to avoid computing the same results again, following the principle of **optimal substructure** and **overlapping subproblems**.

Key Characteristics:

- **Optimal Substructure**: The optimal solution contains optimal solutions to subproblems
- **Overlapping Subproblems**: Same subproblems are solved multiple times
- **Memoization**: Store results to avoid recomputation

Two Approaches:

1. **Top-Down (Memoization)**: Recursive approach with caching
2. **Bottom-Up (Tabulation)**: Iterative approach filling a table

Practice Session on Dynamic Programming (String-based Questions)

1. Longest Common Subsequence (LCS)

Problem: Find the length of the longest common subsequence between two strings.

Approach:

- If characters match, add 1 to LCS of remaining strings
- If characters don't match, take maximum of two possibilities

java

```java
public class LCS {
    // Recursive approach with memoization
    public static int lcsRecursive(String s1, String s2, int i, int j, int[][] memo) {
        if (i == s1.length() || j == s2.length()) {
            return 0;
        }

        if (memo[i][j] != -1) {
            return memo[i][j];
        }

        if (s1.charAt(i) == s2.charAt(j)) {
            memo[i][j] = 1 + lcsRecursive(s1, s2, i + 1, j + 1, memo);
        } else {
            memo[i][j] = Math.max(
                lcsRecursive(s1, s2, i + 1, j, memo),
                lcsRecursive(s1, s2, i, j + 1, memo)
            );
        }

        return memo[i][j];
    }

    // Bottom-up approach
    public static int lcsIterative(String s1, String s2) {
        int m = s1.length();
        int n = s2.length();
        int[][] dp = new int[m + 1][n + 1];

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (s1.charAt(i - 1) == s2.charAt(j - 1)) {
                    dp[i][j] = 1 + dp[i - 1][j - 1];
                } else {
                    dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
                }
            }
        }

        return dp[m][n];
    }
}
```

2. Edit Distance (Levenshtein Distance)

Problem: Find minimum number of operations (insert, delete, replace) to convert one string to another.

Approach:

- If characters match, no operation needed
- If different, try all three operations and take minimum

```java
public class EditDistance {
    public static int minDistance(String word1, String word2) {
        int m = word1.length();
        int n = word2.length();
        int[][] dp = new int[m + 1][n + 1];

        // Initialize base cases
        for (int i = 0; i <= m; i++) {
            dp[i][0] = i; // Delete all characters
        }
        for (int j = 0; j <= n; j++) {
            dp[0][j] = j; // Insert all characters
        }

        // Fill the DP table
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (word1.charAt(i - 1) == word2.charAt(j - 1)) {
                    dp[i][j] = dp[i - 1][j - 1]; // No operation needed
                } else {
                    dp[i][j] = 1 + Math.min(
                        Math.min(dp[i - 1][j],     // Delete
                                 dp[i][j - 1]),     // Insert
                        dp[i - 1][j - 1]          // Replace
                    );
                }
            }
        }

        return dp[m][n];
    }
}
```

3. Longest Palindromic Subsequence

Problem: Find the length of the longest palindromic subsequence in a string.

Approach:

- If characters at both ends match, add 2 to inner substring
- If different, take maximum of excluding either end

```java
public class LongestPalindromicSubsequence {
    public static int longestPalindromeSubseq(String s) {
        int n = s.length();
        int[][] dp = new int[n][n];

        // Every single character is a palindrome of length 1
        for (int i = 0; i < n; i++) {
            dp[i][i] = 1;
        }

        // Check for palindromes of length 2 and more
        for (int len = 2; len <= n; len++) {
            for (int i = 0; i <= n - len; i++) {
                int j = i + len - 1;

                if (s.charAt(i) == s.charAt(j)) {
                    dp[i][j] = 2 + dp[i + 1][j - 1];
                } else {
                    dp[i][j] = Math.max(dp[i + 1][j], dp[i][j - 1]);
                }
            }
        }

        return dp[0][n - 1];
    }
}
```

4. Word Break Problem

Problem: Determine if a string can be segmented into dictionary words.

Approach:

- Use DP to check if substring can be formed using dictionary words

```java
import java.util.*;

public class WordBreak {
    public static boolean wordBreak(String s, List<String> wordDict) {
        Set<String> wordSet = new HashSet<>(wordDict);
        boolean[] dp = new boolean[s.length() + 1];
        dp[0] = true; // Empty string can always be segmented

        for (int i = 1; i <= s.length(); i++) {
            for (int j = 0; j < i; j++) {
                if (dp[j] && wordSet.contains(s.substring(j, i))) {
                    dp[i] = true;
                    break;
                }
            }
        }

        return dp[s.length()];
    }
}
```

5. Distinct Subsequences

Problem: Count the number of distinct subsequences of string `s` that equals string `t`.

Approach:

- If characters match, we can either include or exclude the character
- If different, we can only exclude the character from `s`

```java
public class DistinctSubsequences {
    public static int numDistinct(String s, String t) {
        int m = s.length();
        int n = t.length();
        int[][] dp = new int[m + 1][n + 1];

        // Empty string t can be formed by any string s in one way
        for (int i = 0; i <= m; i++) {
            dp[i][0] = 1;
        }

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (s.charAt(i - 1) == t.charAt(j - 1)) {
                    dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j];
                } else {
                    dp[i][j] = dp[i - 1][j];
                }
            }
        }

        return dp[m][n];
    }
}
```

6. Regular Expression Matching

Problem: Implement wildcard pattern matching with '.' and '*' characters.

Approach:

- Handle different cases for '.', '*', and regular characters

```java
public class RegexMatching {
    public static boolean isMatch(String s, String p) {
        int m = s.length();
        int n = p.length();
        boolean[][] dp = new boolean[m + 1][n + 1];

        dp[0][0] = true; // Empty string matches empty pattern

        // Handle patterns like a*, a*b*, a*b*c*
        for (int j = 2; j <= n; j++) {
            if (p.charAt(j - 1) == '*') {
                dp[0][j] = dp[0][j - 2];
            }
        }

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                char sc = s.charAt(i - 1);
                char pc = p.charAt(j - 1);

                if (pc == '*') {
                    // Check if we can match zero occurrences
                    dp[i][j] = dp[i][j - 2];

                    // Check if we can match one or more occurrences
                    if (p.charAt(j - 2) == sc || p.charAt(j - 2) == '.') {
                        dp[i][j] = dp[i][j] || dp[i - 1][j];
                    }
                } else if (pc == '.' || pc == sc) {
                    dp[i][j] = dp[i - 1][j - 1];
                }
            }
        }

        return dp[m][n];
    }
}
```

Time and Space Complexity Analysis

Common Patterns:

- **String DP Problems**: Usually O(m × n) time and space complexity
- **LCS, Edit Distance**: O(m × n) where m, n are string lengths
- **Palindrome Problems**: $O(n^2)$ for length n string
- **Word Break**: $O(n^2 \times m)$ where n is string length, m is average word length

Space Optimization Tips:

- Use 1D array when only previous row/column is needed
- Use variables when only a few previous values are needed
- Consider the direction of dependency in the DP table

Quiz Questions

1. What is the time complexity of finding the Longest Common Subsequence using dynamic programming? a) O(n) b) O(n log n) c) O(m × n) d) O(2^n)

Answer: c) O(m × n)

2. In the Edit Distance problem, what are the three allowed operations? a) Insert, Delete, Copy b) Insert, Delete, Replace c) Replace, Swap, Delete d) Insert, Replace, Reverse

Answer: b) Insert, Delete, Replace

3. What is the base case for the Longest Palindromic Subsequence problem? a) Empty string has length 0 b) Single character has length 1 c) Two identical characters have length 2 d) All of the above

Answer: b) Single character has length 1

4. In the Word Break problem, what does dp[i] represent? a) Whether substring from 0 to i can be segmented b) Number of ways to segment substring from 0 to i c) Length of longest word ending at position i d) Position of last valid word break

Answer: a) Whether substring from 0 to i can be segmented

5. Which approach is generally more space-efficient for DP problems? a) Top-down with memoization b) Bottom-up with tabulation c) Recursive without memoization d) Both a and b are equally efficient

Answer: b) Bottom-up with tabulation

Practice Coding Questions

1. Longest Common Substring Write a Java method to find the length of the longest common substring between two strings. Unlike subsequence, substring characters must be consecutive.

2. Palindromic Substrings Count Given a string, count how many palindromic substrings it contains. For example, "abc" has 3 palindromic substrings: "a", "b", "c".

3. Interleaving Strings Given three strings s1, s2, and s3, determine whether s3 is formed by interleaving s1 and s2. An interleaved string preserves the relative order of characters from each string.

4. Minimum Window Subsequence Given strings S and T, find the minimum window in S which contains all characters of T as a subsequence. Return the minimum window or empty string if no such window exists.

5. Scramble String Given two strings s1 and s2, determine if s2 is a scrambled string of s1. A scrambled string is obtained by recursively applying a specific scrambling process to a binary tree representation of the string.